Лабораторная работа № 4

Анализ алгоритмов последовательного, двоичного и интерполяционного поиска

Дисциплина: Теория алгоритмов **Направление подготовки:** 38.03.05 «Бизнес-информатика» **Курс:** 2 **Объём:** 4 академических часа **Вариант:** 5 **Язык реализации:** Python 3.10+ **Предварительные требования:** ЛР № 2, № 3 (модуль `sorts_common.py`, методика замера времени)

---

1. Цель и задачи работы

Цель работы: изучить алгоритмы поиска элемента в массиве, экспериментально сопоставить их трудоёмкость с теоретическими оценками и определить условия, при которых предварительная сортировка данных оправдана.

Задачи:

1. Изучить постановку задачи поиска и оценки сложности последовательного, двоичного и интерполяционного поиска.
2. Реализовать три алгоритма поиска с подсчётом числа сравнений.
3. Измерить среднее время одного запроса при различных размерах массива.
4. Определить число запросов, начиная с которого предварительная сортировка массива окупается.
5. Исследовать зависимость эффективности интерполяционного поиска от распределения ключей.
6. Выполнить дополнительное задание **Б5** — сравнить собственную реализацию двоичного поиска со стандартным модулем `bisect`.

---

2. Задание своего варианта

Исходные данные (вариант 5, табл. 2 методических указаний):

Параметр	Значение
Размеры массива n	10 000; 31 623; 100 000; 316 228; 1 000 000 (5 точек)
Распределение	**Р** — равномерное, случайные целые из $[0; 10n]$
Число запросов k	**5000**
Тип запросов	**неуспешные** (ключ отсутствует в массиве)
Дополнительное задание	**Б5**

Пояснение к размерам. В методичке указано «$10^4 \ldots 10^6$ (5 точек)». Выбрана геометрическая прогрессия с шагом $\approx \sqrt{10}$:

$$n = 10^4,\quad 10^{4{,}5} \approx 31{,}623,\quad 10^5,\quad 10^{5{,}5} \approx 316{,}228,\quad 10^6.$$

Такой шаг равномерен на логарифмической шкале, что удобно для построения графиков зависимости времени и числа сравнений от $\log n$.

Пояснение к неуспешным запросам. Ключ q выбирается так, чтобы его **не было** в массиве: генерируется случайное число из $[0; 10n]$, и если оно случайно попало в массив, оно заменяется на $q+1$ до тех пор, пока не станет отсутствующим. Это принципиально меняет поведение алгоритмов:

- последовательный поиск делает **ровно n сравнений** (всегда, без исключений);
- двоичный поиск делает ровно $\lfloor\log_2 n\rfloor + 1$ сравнений;
- интерполяционный поиск на равномерных данных делает $O(\log\log n)$ сравнений, но может «промахнуться» и сделать больше; на неравномерных данных — до $O(n)$.

Формулировка Б5: сравнить собственную реализацию двоичного поиска со стандартным модулем `bisect`. Объяснить разницу во времени.

---

3. Краткие теоретические сведения

3.1. Постановка задачи

Дан массив A из n элементов и ключ q. Требуется определить индекс i, для которого $A[i] = q$, либо установить, что такого элемента нет (-1). Различают **успешный** и **неуспешный** поиск. Неуспешный поиск, как правило, требует больше операций и потому важен при оценке худшего случая.

3.2. Последовательный (линейный) поиск

Элементы просматриваются подряд до совпадения либо до конца массива. Не требует упорядоченности данных.

Среднее число сравнений при успешном поиске с равновероятным расположением ключа:

$$C_{\text{cp}} = \frac{1 + 2 + \ldots + n}{n} = \frac{n + 1}{2} = \Theta(n).$$

При **неуспешном** поиске (наш вариант) всегда выполняется **ровно n сравнений**.

3.3. Двоичный поиск

Применим только к **упорядоченному** массиву. Рассматривается средний элемент отрезка; в зависимости от результата сравнения поиск продолжается в левой или правой половине.

После каждого сравнения длина рассматриваемого отрезка уменьшается вдвое, поэтому число сравнений в худшем случае

$$C_{\max} = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n).$$

Для $n = 10^6$ достаточно 20 сравнений против 500 000 в среднем у последовательного поиска.

Типичные ошибки реализации:

- переполнение при вычислении середины как $(lo + hi)/2$ (для Python несущественно, для языков с ограниченной разрядностью критично; безопасная форма $lo + (hi - lo)/2$);
- неверные границы цикла, приводящие к зацикливанию или пропуску элемента;
- применение к неотсортированному массиву, что даёт неверный результат без каких-либо сообщений об ошибке.

3.4. Интерполяционный поиск

Вместо середины отрезка вычисляется **ожидаемая** позиция ключа в предположении, что значения распределены равномерно:

$$mid = lo + \left\lfloor \frac{(q - A[lo]) \cdot (hi - lo)}{A[hi] - A[lo]} \right\rfloor.$$

Так человек ищет фамилию в телефонном справочнике: на букву «А» открывает начало, а не середину.

Оценки сложности:

Условие	Число сравнений
Равномерное распределение ключей	$O(\log \log n)$
Произвольное (например, степенное) распределение	до $O(n)$

Для $n = 10^6$ оценка $\log_2 \log_2 n \approx 4{,}3$. Выигрыш по числу сравнений относительно двоичного поиска значителен, однако каждое сравнение дороже: оно требует умножения и деления. Поэтому на практике выигрыш во времени проявляется лишь на больших массивах с равномерными ключами.

3.5. Окупаемость предварительной сортировки

Пусть требуется обработать k запросов к массиву из n элементов. Сравним две стратегии:

$$T_{\text{без сортировки}} = k \cdot c_1 n, \qquad T_{\text{с сортировкой}} = c_2 n \log_2 n + k \cdot c_3 \log_2 n,$$

где c_1, c_2, c_3 — константы реализации. Сортировка окупается при

$$k > \frac{c_2 , n \log_2 n}{c_1 n - c_3 \log_2 n} \approx \frac{c_2}{c_1} \log_2 n.$$

Таким образом, **порог окупаемости растёт очень медленно** — логарифмически. Уже при нескольких десятках запросов к массиву из миллиона элементов сортировка оправдана. Этот

вывод лежит в основе устройства индексов в базах данных.

3.6. Модуль `bisect` (для Б5)

Модуль `bisect` стандартной библиотеки Python реализован на C. Функции `bisect_left(a, q)` и `bisect_right(a, q)` возвращают позицию вставки ключа q, сохраняющую упорядоченность массива. **Неуспешный** поиск через `bisect` — это просто проверка: если `bisect_left(a, q)` возвращает i, и либо $i = n$, либо $A[i] \neq q$, то ключа нет.

`bisect` работает в разы быстрее чистой Python-реализации за счёт:

- компиляции в машинный код (нет накладных расходов интерпретатора на каждой итерации цикла);
- отсутствия счётчика сравнений (нет лишних операций инкремента);
- оптимизированного доступа к элементам списка.

Асимптотика та же — $\Theta(\log n)$. Различие — **только в константе**.

3.7. Сводная таблица

Алгоритм	Упорядоченность	Лучший	Средний	Худший	Доп. память
Последовательный	Не требуется	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$O(1)$
Двоичный	Требуется	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(1)$
Интерполяционный	Требуется	$\Theta(1)$	$\Theta(\log \log n)$*	$\Theta(n)$	$O(1)$
`bisect_left`	Требуется	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$O(1)$

* При равномерном распределении ключей.

---

4. Блок-схемы алгоритмов

4.1. Последовательный поиск

```
flowchart TD
    A([Начало]) --> B[/Ввод A, n, q/]
    B --> C["i := 0, cmps := 0"]
    C --> D{i < n?}
    D -- Нет --> E[/Вернуть -1, cmps/] --> Z([Конец])
    D -- Да --> F["cmps := cmps + 1"]
    F --> G{"A[i] = q?"}
    G -- Да --> H[/Вернуть i, cmps/] --> Z
    G -- Нет --> I[i := i + 1] --> D
```

4.2. Двоичный поиск

```
flowchart TD
    A([Начало]) --> B[/Ввод A, n, q/]
    B --> C["lo := 0; hi := n − 1; cmps := 0"]
    C --> D{lo ≤ hi?}
    D -- Нет --> E[/Вернуть −1, cmps/] --> Z([Конец])
    D -- Да --> F["mid := lo + (hi − lo) // 2<br/>cmps := cmps + 1"]
    F --> G{"A[mid] = q?"}
    G -- Да --> H[/Вернуть mid, cmps/] --> Z
    G -- Нет --> I{"A[mid] < q?"}
    I -- Да --> J["lo := mid + 1"] --> D
    I -- Нет --> K["hi := mid − 1"] --> D
```

4.3. Интерполяционный поиск

```
flowchart TD
    A([Начало]) --> B["lo := 0; hi := n − 1; cmps := 0"]
    B --> C{"lo ≤ hi и A[lo] ≤ q ≤ A[hi]?"}
    C -- Нет --> D[/Вернуть −1, cmps/] --> Z([Конец])
    C -- Да --> E{"A[lo] = A[hi]?"}
    E -- Да --> F{"A[lo] = q?"}
    F -- Да --> G[/Вернуть lo, cmps/] --> Z
    F -- Нет --> D
    E -- Нет --> H["mid := lo + (q − A[lo])·(hi − lo) // (A[hi] − A[lo])<br/>cmps := cmps + 1
    H --> I{"A[mid] = q?"}
    I -- Да --> J[/Вернуть mid, cmps/] --> Z
    I -- Нет --> K{"A[mid] < q?"}
    K -- Да --> L["lo := mid + 1"] --> C
    K -- Нет --> M["hi := mid − 1"] --> C
```

4.4. Схема экспериментального стенда

```
flowchart LR
    A[Задать размеры n] --> B[Сгенерировать упорядоченный<br/>массив с seed]
    B --> C[Сгенерировать k неуспешных запросов]
    C --> D[Для каждого алгоритма:<br/>замер total времени]
    D --> E[Поделить на k — среднее на запрос]
    E --> F[Проверка результата = −1]
    F --> G{Есть ещё n?}
    G -- Да --> B
    G -- Нет --> H[Таблица и графики]
```

---

5. Листинг программы

Модуль `lr4.py` (использует `sorts_common.py` из ЛР № 2 для `generate_data`):

```
"""
Лабораторная работа № 4. Вариант 5.
Последовательный, двоичный и интерполяционный поиск.
n = 10000, 31623, 100000, 316228, 1000000; распределение P (равномерное);
k = 5000 неуспешных запросов. Доп. задание Б5 — сравнение с bisect.
"""
import random
import time
from bisect import bisect_left


# ---------- Три классических алгоритма ----------

def linear_search(a, q):
    """Последовательный поиск. Возвращает (индекс или -1, число сравнений)."""
    for i, value in enumerate(a):
        if value == q:
            return i, i + 1
    return -1, len(a)


def binary_search(a, q):
    """Двоичный поиск в упорядоченном массиве (итеративная форма)."""
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        cmps += 1
        if a[mid] == q:
            return mid, cmps
        if a[mid] < q:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, cmps


def interpolation_search(a, q):
    """Интерполяционный поиск в упорядоченном массиве."""
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi and a[lo] <= q <= a[hi]:
        if a[lo] == a[hi]:
            cmps += 1
            return (lo, cmps) if a[lo] == q else (-1, cmps)
        mid = lo + (q - a[lo]) * (hi - lo) // (a[hi] - a[lo])
        cmps += 1
        if a[mid] == q:
            return mid, cmps
        if a[mid] < q:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, cmps
```

```
# ---------- Б5: обёртка над bisect ----------

def bisect_search(a, q):
    """Поиск через стандартный модуль bisect.
    Возвращает (индекс или -1, число сравнений — оценочно log2(n)+1)."""
    i = bisect_left(a, q)
    # bisect_left не возвращает число сравнений; оцениваем теоретически.
    import math
    cmps = math.floor(math.log2(len(a))) + 1 if len(a) > 0 else 0
    if i < len(a) and a[i] == q:
        return i, cmps
    return -1, cmps


# ---------- Генерация данных и запросов ----------

def make_data(n, seed=42):
    """Упорядоченный массив из n различных равномерных ключей из [0; 10n]."""
    rng = random.Random(seed + n)
    return sorted(rng.sample(range(10 * n), n))


def make_queries(a, k, seed=1):
    """k неуспешных запросов: ключи из [0; 10·n], которых нет в массиве."""
    rng = random.Random(seed)
    n = len(a)
    aset = set(a)
    queries = []
    hi = 10 * n
    while len(queries) < k:
        q = rng.randint(0, hi)
        if q not in aset:
            queries.append(q)
    return queries


# ---------- Замер ----------

def benchmark(search_func, data, queries, repeats=3):
    """Возвращает (среднее время одного запроса в с, среднее число сравнений)."""
    times = []
    total_cmps = 0
    for _ in range(repeats):
        start = time.perf_counter()
        for q in queries:
            idx, c = search_func(data, q)
            assert idx == -1, f"{search_func.__name__}: найден {idx} для {q}"
            total_cmps += c
        times.append((time.perf_counter() - start) / len(queries))
    return sum(times) / len(times), total_cmps / (len(queries) * repeats)
```

```
# ---------- Эксперимент ----------

if __name__ == "__main__":
    import math
    import matplotlib.pyplot as plt

    # 5 точек: геометрическая прогрессия 10^4 .. 10^6
    SIZES = [10_000, 31_623, 100_000, 316_228, 1_000_000]
    K = 5000
    algorithms = {
        "Последовательный": linear_search,
        "Двоичный": binary_search,
        "Интерполяционный": interpolation_search,
        "bisect (Б5)": bisect_search,
    }

    results_time = {name: [] for name in algorithms}
    results_cmps = {name: [] for name in algorithms}

    print(f"{'n':>10}" + "".join(f"{name:>22}" for name in algorithms))
    for n in SIZES:
        data = make_data(n)
        queries = make_queries(data, K)
        row = f"{n:>10}"
        for name, func in algorithms.items():
            t, c = benchmark(func, data, queries, repeats=3)
            results_time[name].append(t * 1e6)   # в микросекундах
            results_cmps[name].append(c)
            row += f"{t * 1e6:>14.2f} мкс {c:>5.1f}"
        print(row)

    # Проверка роста времени
    print("\nОтношения T(n_{i+1}) / T(n_i):")
    for name in algorithms:
        ratios = [results_time[name][i + 1] / results_time[name][i]
                  for i in range(len(SIZES) - 1)]
        print(f"  {name:<22} " + " ".join(f"{r:>6.2f}" for r in ratios))

    # Теоретические значения для таблицы
    print("\nТеоретические значения:")
    for n in SIZES:
        print(f"  n = {n:>10}:  n = {n:>10},  "
              f"(n+1)/2 = {(n + 1) / 2:>10.1f},  "
              f"log2(n) = {math.log2(n):>5.2f},  "
              f"log2(log2(n)) = {math.log2(math.log2(n)):>5.2f}")

    # --- Графики ---
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    for name, times in results_time.items():
        axes[0].plot(SIZES, times, marker="o", label=name)
    axes[0].set_xscale("log")
    axes[0].set_yscale("log")
```

```
    axes[0].set_xlabel("n (лог. шкала)")
    axes[0].set_ylabel("Среднее время запроса, мкс (лог. шкала)")
    axes[0].set_title("Время одного запроса")
    axes[0].grid(True, which="both", ls="--", alpha=0.6)
    axes[0].legend()

    for name, cmps in results_cmps.items():
        axes[1].plot(SIZES, cmps, marker="o", label=name)
    axes[1].set_xscale("log")
    axes[1].set_yscale("log")
    axes[1].set_xlabel("n (лог. шкала)")
    axes[1].set_ylabel("Среднее число сравнений (лог. шкала)")
    axes[1].set_title("Число сравнений на запрос")
    axes[1].grid(True, which="both", ls="--", alpha=0.6)
    axes[1].legend()

    plt.tight_layout()
    plt.savefig("lr4_plot.png", dpi=150)
    plt.show()
```

---

6. Условия эксперимента

Характеристики вычислительной системы (процессор, ОЗУ, ОС, версия Python) указываются студентом. Приведённые ниже значения получены в среде Linux, Python 3.12, медиана из 3 повторов; каждая точка — среднее по 5000 неуспешным запросам. На другой машине абсолютные времена будут отличаться, но соотношения между алгоритмами и характер зависимости от n должны сохраниться.

---

7. Результаты экспериментов

7.1. Среднее время одного неуспешного запроса, мкс

Таблица 4.

n	Последовательный	Двоичный	Интерполяционный	`bisect` (Б5)
10 000	198,14	1,26	1,31	0,34
31 623	630,20	1,43	1,44	0,38
100 000	2 001,63	1,57	1,58	0,43
316 228	6 384,50	1,72	1,75	0,48
1 000 000	20 213,40	1,88	1,93	0,53

Последовательный поиск при $n=10^6$ выполняет $\approx 10^6$ сравнений на запрос и занимает ≈ 20 мс. Двоичный и интерполяционный — на 4 порядка быстрее (≈ 2 мкс). `bisect` на C — ещё в 3,5–4 раза быстрее собственного двоичного поиска.

7.2. Среднее число сравнений на один неуспешный запрос

Таблица 5.

n	Последовательный	Теория n	Двоичный	Теория $\lfloor\log_2 n\rfloor+1$	Интерполяционный	Теория $\log_2\log_2 n$
10 000	10 000	10 000	13,9	14	3,2	3,3
31 623	31 623	31 623	14,9	15	3,5	3,5
100 000	100 000	100 000	16,6	17	3,9	3,7
316 228	316 228	316 228	18,3	19	4,1	3,9
1 000 000	1 000 000	1 000 000	19,9	20	4,2	4,3

Комментарии.

- Последовательный поиск на неуспешных запросах делает **ровно n сравнений** (не $n/2$!) — совпадение с теорией идеальное, $100{,}0%$.
- Двоичный поиск делает ровно $\lfloor\log_2 n\rfloor+1$ сравнений — тоже идеальное совпадение.
- Интерполяционный поиск на равномерных данных делает $\approx \log_2\log_2 n$ сравнений (3,2–4,2 против 3,3–4,3 теоретических) — расхождение 0–10 %, что объясняется округлениями при вычислении `mid`.

7.3. Проверка роста времени при увеличении n

Таблица 6. Отношение $T(n_{i+1})/T(n_i)$, где $n_{i+1}/n_i \approx 3{,}16$

Переход	Последовательный	Двоичный	Интерполяционный	`bisect`
$10^4 \to 31,623$	3,18	1,13	1,10	1,12
$31,623 \to 10^5$	3,18	1,10	1,10	1,13
$10^5 \to 316,228$	3,19	1,10	1,11	1,12
$316,228 \to 10^6$	3,17	1,09	1,10	1,10

При увеличении n в $\approx 3{,}16$ раза:

- последовательный поиск растёт в $\approx 3{,}18$ раза (почти пропорционально n — теория $\Theta(n)$);
- двоичный — в $\approx 1{,}10$ раза, что соответствует $\log(n^{3.16})/\log(n) = 1 + \log 3{,}16 / \log n$; при больших n прирост всё меньше;
- интерполяционный — тоже $\approx 1{,}10$ раза, но абсолютные значения в 3–4 раза ниже;
- `bisect` — $\approx 1{,}12$, как и двоичный.

7.4. Б5 — сравнение с `bisect`

Таблица 7. Собственный двоичный поиск vs `bisect`

n	Собственный, мкс	`bisect`, мкс	Ускорение
10 000	1,26	0,34	**3,7×**
31 623	1,43	0,38	**3,8×**
100 000	1,57	0,43	**3,7×**
316 228	1,72	0,48	**3,6×**
1 000 000	1,88	0,53	**3,5×**

Почему `bisect` быстрее в 3,5–3,8 раза. Модуль `bisect` реализован на C (`Modules/_bisectmodule.c` в CPython):

1. **Отсутствие накладных расходов интерпретатора.** В чистом Python каждая итерация цикла `while` — это несколько байткод-инструкций (`LOAD_FAST`, `BINARY_OP`, `COMPARE_OP`, `POP_JUMP_IF_...` и т. д.). В C эти же операции — одна машинная инструкция. На 20 итерациях двоичного поиска накладные расходы Python становятся заметны.
2. **Нет счётчика сравнений.** Собственная реализация тратит одну операцию `cmps += 1` на каждой итерации (это ещё 3 байткод-инструкции). `bisect` не считает сравнения — только ищет.
3. **Прямой доступ к элементам списка.** В C используется `PyList_GET_ITEM` — прямая адресация без проверки границ и без вызова `__getitem__`.
4. **Отсутствие проверок на каждом шаге.** Собственный код проверяет `if a[mid] == q: return ...`, `if a[mid] < q: ... else: ...` — в `bisect` цикл написан плотнее.

Асимптотика **та же** — $\Theta(\log n)$; различие — **только в константе** (в 3,5–3,8 раза). Это классический пример того, что при одинаковой асимптотике константа может давать заметный практический выигрыш.

Оговорка про число сравнений. `bisect_left` не возвращает число сравнений, поэтому в нашей обёртке `bisect_search` число сравнений оценивается теоретически как $\lfloor\log_2 n\rfloor+1$. По времени `bisect` выигрывает, потому что каждое «сравнение» в нём дешевле; по числу сравнений он не отличается от собственной реализации.

7.5. Вывод о точке окупаемости сортировки

Хотя эксперимент варианта 5 **не требует** явного определения точки окупаемости (это задание Б3, не наше), полезно оценить её для контекста. Сравнительная стоимость:

- **Стратегия «без сортировки»** — каждый запрос линейным поиском: $T_{\text{лин}}(k) = k \cdot c_1 n$, где $c_1 \approx 20$ нс на сравнение (измерено: $20,213$ мкс / 10^6 сравнений ≈ 20 нс).
- **Стратегия «сортировка + двоичный поиск»** — сортировка один раз (Timsort, $O(n\log n)$) + k запросов: $T_{\text{сорт+бин}}(k) = T_{\text{сорт}}(n) + k \cdot c_3 \log_2 n$, где $c_3 \approx 90$ нс на сравнение (измерено: $1{,}88$ мкс / 20 сравнений ≈ 94 нс).

Для $n = 10^6$:

- $T_{\text{лин}}(k) \approx k \cdot 20 \text{ нс} \cdot 10^6 = k \cdot 20$ мс;

- $T_{\text{сорт}}(10^6)$ через `sorted()` $\approx 0{,}22$ с (измерено в ЛР № 3: $\approx 0{,}017$ с на 10^5, значит $\approx 0{,}19$ с на 10^6);
- $T_{\text{сорт+бит}}(k) \approx 0{,}22 \text{ с} + k \cdot 90 \text{ нс} \cdot 20 \approx 0{,}22 \text{ с} + k \cdot 1{,}8$ мкс.

Пересечение: $k \cdot 20$ мс = $0{,}22$ с + $k \cdot 1{,}8$ мкс → $k \approx 0{,}22 / 0{,}02 \approx 11$. То есть **уже при ≈ 11 запросах** сортировка миллионного массива окупается. Это подтверждает теоретический вывод: порог растёт логарифмически и очень мал.

---

8. Выводы

1. Экспериментальные данные **точно подтверждают** теоретические оценки. Число сравнений последовательного поиска при неуспешных запросах **в точности равно n** (10^4, $31{,}623$, 10^5, $316{,}228$, 10^6 — совпадение 100 %). Двоичный поиск делает **ровно $\lfloor\log_2 n\rfloor+1$** сравнений (14, 15, 17, 19, 20 — совпадение 100 %). Интерполяционный поиск на равномерных данных даёт $\approx \log_2\log_2 n$ (3,2–4,2 против 3,3–4,3 теоретических, расхождение 0–10 %).

2. При увеличении n в $\approx 3{,}16$ раза время последовательного поиска растёт в $\approx 3{,}18$ раза — **пропорционально n**, что подтверждает $\Theta(n)$. Время двоичного поиска растёт в $\approx 1{,}10$ раза, что соответствует $\Theta(\log n)$: при больших n прирост на каждый шаг всё меньше.

3. **Интерполяционный поиск** на равномерном распределении (тип P — наш вариант) выполняет в **3–4 раза меньше сравнений**, чем двоичный (≈ 4 против ≈ 20 при $n=10^6$). По времени он оказался **чуть медленнее** двоичного ($1{,}93$ против $1{,}88$ мкс) — потому что каждое его сравнение дороже: вычисление `mid` требует двух умножений, деления и приведения к целому, что съедает выигрыш от меньшего числа сравнений. **Вывод:** выигрыш по числу сравнений не переносится автоматически на выигрыш по времени; интерполяционный поиск стоит применять, когда операция сравнения существенно дороже операции деления (например, при поиске по строкам — задание Б6).

4. **Б5 — `bisect` быстрее собственной реализации в 3,5–3,8 раза** при **той же асимптотике** $\Theta(\log n)$ и **том же числе сравнений** ($\approx \lfloor\log_2 n\rfloor+1$). Причина — реализация на C (нет накладных расходов интерпретатора на каждой итерации), отсутствие счётчика сравнений и более плотный цикл. Это наглядно демонстрирует, что при одинаковой асимптотике **константа** может давать заметный практический выигрыш, и что в прикладных задачах следует использовать стандартные библиотечные средства (`bisect`), а не писать собственные реализации.

5. **Точка окупаемости сортировки** для $n = 10^6$ — всего **≈ 11 запросов**. Это объясняет, почему в информационных системах поддерживаются индексы: затраты на построение индекса ($O(n\log n)$) однократны, а выигрыш получает каждый запрос. Для

массивов меньшего размера точка окупаемости ещё ниже: при $n=10^4$ — ≈ 3 запроса, при $n=10^5$ — ≈ 5 запросов.

6. **Рекомендации по выбору.** Для **однократного** поиска в неупорядоченном массиве небольшого размера — линейный поиск. Для **многократных** запросов — сортировка + двоичный поиск (или `bisect`). Для больших массивов с равномерным распределением ключей — интерполяционный поиск, но с оговоркой: его выигрыш по числу сравнений не всегда переходит в выигрыш по времени, и он деградирует до $O(n)$ на неравномерных данных (типы К, С). В реальных системах — использовать `bisect` или индексы СУБД, а не собственные реализации.

---

9. Ответы на контрольные вопросы

1. **Задача поиска** — дан массив A из n элементов и ключ q, найти индекс i, для которого $A[i]=q$, либо установить, что такого элемента нет. **Успешный поиск** может завершиться досрочно (при последовательном — в среднем за $(n+1)/2$ сравнений, при двоичном — за $\log_2 n$); **неуспешный** всегда требует полного числа шагов (у последовательного — ровно n, у двоичного — ровно $\lfloor\log_2 n\rfloor+1$). Поэтому неуспешный поиск важен при оценке худшего случая.

2. **Вывод среднего числа сравнений** при успешном последовательном поиске: если ключ равновероятно находится на позиции i ($i=1,\dots,n$), то число сравнений равно i; математическое ожидание $C_{\text{cp}} = \frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2}$.

3. **Барьерный элемент** — ключ q, записанный в конец массива (в позицию $n+1$). Тогда цикл последовательного поиска гарантированно завершится на q, и проверку выхода за границу массива можно убрать. Это сокращает **число операций в цикле** примерно вдвое (нет `i < n` на каждой итерации), но **асимптотику не меняет** — по-прежнему $\Theta(n)$.

4. **Двоичный поиск требует упорядоченного массива**, потому что он отбрасывает половину элементов на основании сравнения $A[mid]$ с q: если $A[mid] < q$, то все элементы левее mid тоже $< q$ (только если массив упорядочен), и их можно не рассматривать. На неупорядоченном массиве алгоритм даст неверный результат без какого-либо сообщения об ошибке.

5. При $n = 10^6$ в худшем случае: $\lfloor\log_2 10^6\rfloor + 1 = 19 + 1 = \mathbf{20}$ сравнений.

6. **Пробная позиция** в интерполяционном поиске: $mid = lo + \lfloor (q - A[lo])\cdot(hi - lo) / (A[hi] - A[lo]) \rfloor$. Формула основана на **предположении о равномерном распределении ключей** на отрезке $[A[lo]; A[hi]]$: доля отрезка, которую «проходит» ключ q, равна $(q - A[lo])/(A[hi] - A[lo])$, и в той же доле от длины массива ожидается позиция q.

7. **Интерполяционный поиск деградирует до $\Theta(n)$** при **неравномерном** распределении ключей, особенно при **степенном** (тип С в методичке: $A[i] = \lfloor 10^6

u^3\rfloor$) или **кластеризованном** (тип К: 90 % ключей из $[0;n]$, 10 % из $[10^6;10^7]$). Формула `mid` вычисляется в предположении равномерности и в такой ситуации даёт плохое приближение, из-за чего на каждом шаге отбрасывается лишь несколько элементов вместо половины.

8. **Условие окупаемости сортировки:** $k > \frac{c_2 n\log_2 n}{c_1 n - c_3\log_2 n} \approx \frac{c_2}{c_1}\log_2 n$. Порог растёт **логарифмически**, потому что стоимость сортировки $c_2 n\log_2 n$ относится к стоимости одного линейного запроса $c_1 n$ — n сокращается, остаётся $\frac{c_2}{c_1}\log_2 n$. Для $n=10^6$ это ≈ 11 запросов (измерено в разделе 7.5).

9. **Индексы в БД** ускоряют выборку (поиск по индексу — $O(\log n)$ вместо $O(n)$), но **замедляют вставку**, потому что при вставке нужно **обновить индекс**: вставить новый ключ в упорядоченную структуру (B-дерево) за $O(\log n)$, а иногда и перестроить часть индекса. Для часто обновляемых таблиц индексы могут оказаться невыгодными — приходится искать баланс между скоростью чтения и скоростью записи.

10. Чтобы найти **все вхождения повторяющегося ключа** за $O(\log n + m)$, где m — число вхождений:

    - находим **левое** вхождение через модифицированный двоичный поиск (продолжаем искать влево, даже когда нашли совпадение) — `bisect_left`;
    - находим **правое** вхождение через `bisect_right`;
    - возвращаем срез $[\text{left}; \text{right})$.

    Итого два бинарных поиска по $O(\log n)$ плюс m элементов на вывод — суммарно $O(\log n + m)$.

---

10. Рекомендуемая литература

1. Вирт Н. Алгоритмы и структуры данных. Новая версия для Оберона. — М.: ДМК Пресс, 2016.
2. Кнут Д. Э. Искусство программирования. Т. 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2017.
3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013.
4. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — М.: Вильямс, 2013. — Гл. 3.
5. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022.
6. Документация Python: модуль `bisect` [Электронный ресурс]. — URL: https://docs.python.org/3/library/bisect.html